

COURSE Machine Learning on Graphs
INSTRUCTOR Matteo Zignani matteo.zignani@unimi.it Computer Science Dept. room 7016 or Connects Lab (room 7017)
CFU 6 CFU ↳ 7th floor
IF HOUR 48 h
TYPE Master Degree in CS - Pillar 1 / Complementary course
SCHEDULE 13/04/2026 → 18/06/2026

3Q
Monday 13:30 - 16:30 → 13:30 → 14:45 + 15:00 → 16:00 Room 403 - Settore Didattico
Thursday 09:30 - 12:30 → 09:30 → 10:45 + 11:00 → 12:00 Room 505 - via Valvasoni Perani
• CANCELLED 20/04 - 23/04 - 11/06
• Thursday 16/04 → 10:30 - 13:30 (10:30 → 11:45 + 12:00 → 13:00) Room B Physics Dept.

HOW TO CONTACT ME : mail or MS Teams by chat office hour by appointment
↓
use [MLG26] as prefix of email subject introduce yourself in the first message of the chat

PROGRAM **Introduction to Machine Learning on Graphs**
- Fundamentals of graph theory: basic definitions (nodes, edges, degree, paths, connectivity, centrality), graph types (directed, weighted, multigraphs, bipartite, etc.), computational representations (adjacency lists, matrices).
- Standard terminology and notation used in the literature and implementations.
Machine Learning on Graphs without Feature Learning
- Main tasks:
- Topology-based methods
- Graph Kernels and Graph SVM
- Graph clustering techniques
Graph Representation Learning with Shallow Networks
- Motivations for graph embedding: reducing structural complexity, mapping nodes/edges/graphs to vector spaces.
- Shallow methods (non end-to-end learning)
- Evaluation of embeddings
Graph Representation Learning with Deep Feature Learning
- Introduction to Graph Neural Networks (GNNs)
- Main architectures
- Advanced training aspects
- Implementation and evaluation:
Graph Representation Learning for Heterogeneous Networks and Knowledge Graphs
- Definition of heterogeneous networks: graphs with multiple node and edge types.
- Knowledge graphs: graphs with typed relations and explicit semantics (subject-predicate-object triples).
- Embedding methods for heterogeneous networks:
- Knowledge graph embedding methods:
- Applications
Graph Representation Learning for Temporal Networks
- Introduction to temporal graphs:
- Methods for discrete-time temporal graphs:
- Methods for continuous-time dynamic graphs:
- Case studies

REQUISITES : not official but "Graph Theory", course on ML (intro) help
TEACHING MATERIALS : personal notes + Jupyter notebooks release on MyAriel + recorded lectures
↓
myariel.unimi.it/course/view.php?id=9462
REFERENCE BOOK: "Graph Representation Learning" @2020 by William Hamilton → www.cs.mcgill.ca/~wch/gre_book
TOOLS : coding with Python, PyTorch Geometric - PyG-, NetworkX
EXAM : discussion on a research paper on ML on graph / discussion is scheduled (free to select the day) but registration follows the calendar
method + coding / ≥ 30 minutes / oral / individual

• **GRAPH** $G = (V, E)$

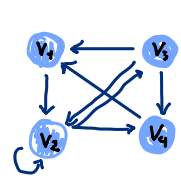
pair of disjoint sets $V = \{v_i\}$ set of nodes $V(G)$

$E \subseteq V^2 = V \times V$ set of edges $E(G)$

constraints on selecting elements from $V \times V$

a) $E = \{(u, v) \in V^2 \mid u, v \in E \Leftrightarrow v, u \in E\} \Rightarrow$ UNORDERED PAIRS

b) $\forall v \in V \quad (v, v) \notin E \Rightarrow$ NO SELF-LOOPS



	V_1	V_2	V_3	V_4
V_1		V_1V_2	V_1V_3	V_1V_4
V_2	V_2V_1		V_2V_3	V_2V_4
V_3	V_3V_1	V_3V_2		V_3V_4
V_4	V_4V_1	V_4V_2	V_4V_3	

Can be added artificially in ML

SIMPLE GRAPH = undirected + no self-loops

UNDIRECTED GRAPH

$E = \{\{u, v\} \mid u, v \in V, u \neq v\}$

if we remove constraints $\Rightarrow$ **DIRECTED GRAPH**

asymmetry in the relationships $\Rightarrow$ direction of links

$\Rightarrow$ SEMANTIC ORIENTATION:

- causality
- temporality
- dependence
- citation
- transfer
- flow

role of nodes in a link

- source
- number
- head
- subject
- target
- destination
- receiver
- tail
- object

(u, v) or $\{u, v\}$

u, v = endpoints

= adjacent vertices in G

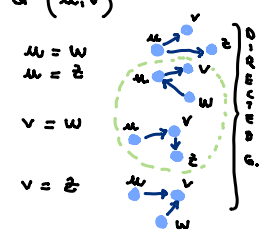
= neighboring vertices in G

= incident with edge $\{u, v\}$ or (u, v)

(u, v) adjacent if they have exactly ONE COMMON ENDPOINT

$\{u, v\}$ adjacent if they have exactly ONE COMMON ENDPOINT

undirected g.



graph as a set of links

$E = \{(u, v) \mid u, v \in V\}$

• **ORDER** of $G = |V| = m$ # of vertices

SIZE of $G = |E| = \ell$ # of edges or links

DENSITY of G ratio between size and all the possible links among m vertices

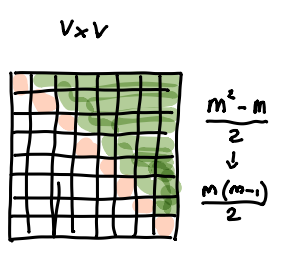
property to indicate

SPARSE GRAPH $\ell = O(m)$

DENSE GRAPH $\ell = O(m^2)$

computationally and modelling implications $\Rightarrow$ sparse data structure

undirected g. $\text{dens}(G) = \frac{2\ell}{m(m-1)}$



directed g. $\text{dens}(G) = \frac{\ell}{m(m-1)}$

$m^2 - m = m(m-1)$

• in digraph, **RECIPROCITY** of G $\approx$ prob. observing (v, u) given $e = (u, v)$

RECIPROCAL LINK

$\text{rec}(G) = \frac{\ell_{\leftrightarrow}}{\ell}$ (# reciprocal links)

• you can derive different graphs starting from G and G'

Δ **COMPLEMENT** of $G = \{(u, v) \in V(G)^2 \mid (u, v) \notin E(G)\} \approx$ **REVERSE GRAPH** of G

$\{(u, v) \mid u, v \in V(G) \wedge \{u, v\} \notin E(G)\}$

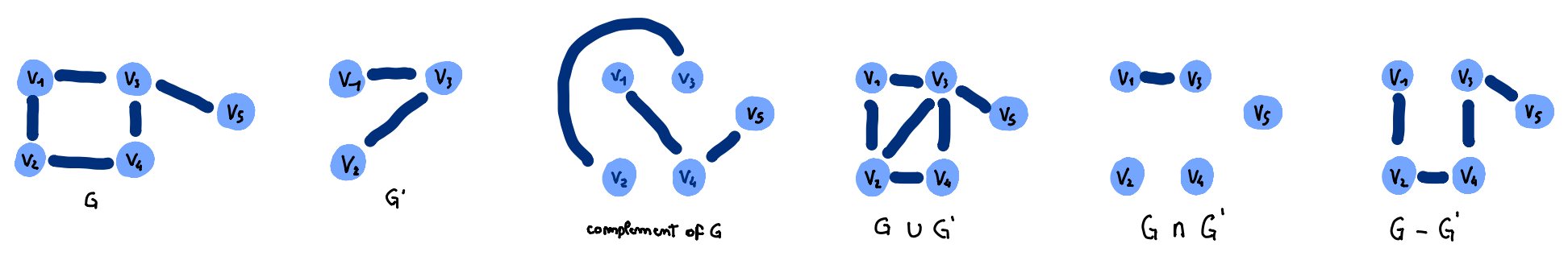
Δ **UNION** $G \cup G' \Rightarrow E(G) \cup E(G')$

Δ **INTERSECTION** $G \cap G' \Rightarrow E(G) \cap E(G')$

Δ **DIFFERENCE** $G \setminus G'$ or $G - G' \Rightarrow E(G) - E(G')$

all possible links among nodes in G that are not in $E(G)$

set operations on $E(G)$



Δ **LINE GRAPH** of $G \rightarrow e \in E(G)$ is a node in $L(G)$

$\sim$ for high order relationships, overlapping communities, graph similarity

undir $V(L(G)) = E(G)$

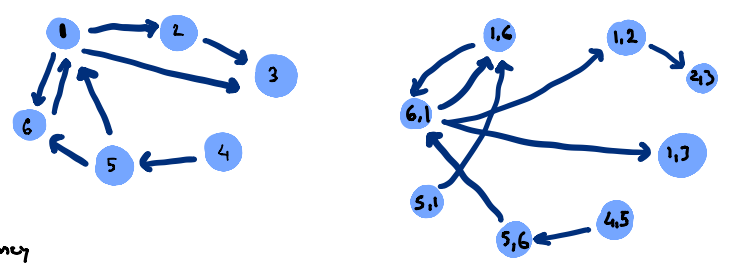
directed $V(L(G)) = E(G)$

$\{(u, v) = a, (w, z) = b\} \in E(L(G)) \Leftrightarrow v = w$

two links can be traversed consecutively

$u \rightarrow v \rightarrow w$

specific class of link adjacency for directed graph



• **EDGE-ATTRIBUTED GRAPH** $\rightarrow$ relationships carry more info than only binary relation (e or $\&$)

- distance
- strength
- capacity
- timestamp
- probability
- similarity
- frequency
- relation type
- cost
- correlation

introduce a set of functions $\{f_1, \dots, f_{d_e}\}$ s.t. $f_i: E \rightarrow \mathbb{S}_i$

ATTRIBUTE FUNCTIONS

generic set of values

$\rightarrow \mathbb{S}_i = \mathbb{R}$ assign to e a vector $\vec{e} \in \mathbb{R}^{d_e}$

$\rightarrow$ only $f_1 = w: E \rightarrow \mathbb{R}$ **WEIGHT FUNCTION**

stacking $\vec{e}$ for all links $\Rightarrow E \in \mathbb{R}^{\ell \times d_e}$

EDGE FEATURE MATRIX

• **NODE-ATTRIBUTED GRAPH** → enrich nodes with contextual / semantic info

- node type
- timestamp
- text / documents
- numerical values
- images
- categorical values

introduce a set of functions $\{f_1, \dots, f_{d_v}\}$ s.t. $f_i: E \rightarrow S_i$
generic set of values $S_i = \mathbb{R}$ assign to e a vector $\vec{u} \in \mathbb{R}^{d_v}$

$[u_1, \dots, u_{d_v}]$
stacking $\vec{u}$ for all links $\Rightarrow U \in \mathbb{R}^{m \times d_v}$
NODE FEATURE MATRIX

attributes a set of functions easily fit into associative memory data structures, s.a. dictionary, maps

$$\begin{aligned} \{f_1: E \rightarrow S_1, \dots, f_d: E \rightarrow S_d\} \\ \theta = (u, v) = \{ \begin{aligned} &"f_1": \langle \text{value in } S_1 \rangle, \\ &\vdots \\ &"f_m": \langle \text{value in } S_d \rangle \end{aligned} \end{aligned}$$

e.g. $(u, v)[f_1] = 4.75$

• **REPRESENTING GRAPH**

Δ **ADJACENCY MATRIX** → inspired by $E \subseteq V^2$

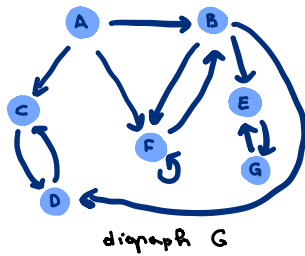
$$A \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|} \text{ s.t. } A_{u,v} = 1 \Leftrightarrow (u, v) \in E$$

squared matrix

• for undir. graph A is symmetric $A = A^T$
 $(u, v) \in E \Leftrightarrow (v, u) \in E$



• for dir. graph the symmetry holds for reciprocal links only, e.g. $c \rightarrow d$ and $d \rightarrow c$



	A	B	C	D	E	F	G	
A	0	1	1	0	0	1	0	3
B	0	0	0	1	1	1	0	3
C	0	0	0	1	0	0	0	1
D	0	0	1	0	0	0	0	1
E	0	0	0	0	0	0	1	1
F	0	1	0	0	0	1	0	2
G	0	0	0	0	1	0	0	1
	0	2	2	2	2	3	1	12

$\sum_{v=1}^{|\mathcal{V}|} A_{u,v}$ row-wise summation

$\sum_{u=1}^{|\mathcal{V}|} A_{u,v}$ column-wise summation

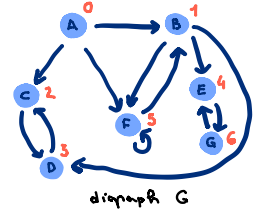
A is good for dense graph but waste of memory to store "0": K = bytes for a single element $\Rightarrow m^2 K$ for A

$\mathcal{O}(K)$ for useful info in sparse graph $\mathcal{O}(m) \approx m \cdot K$

if $K=8$ (double in C) $\rightarrow 7^2 \cdot 8 = 392$
 $12 \cdot 2 \cdot 8 = 196$
store (u, v) = position of "1"s

format for **STARSE (ADJACENCY) MATRIX**

• **Coordinate List - COO**
set of entry (row, column, value)



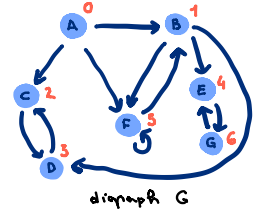
A	A	B	B	B	C	D	E	F	F	G
B	C	F	F	D	E	D	C	G	F	B
1	1	1	1	1	1	1	1	1	1	1

0	0	1	1	2	3	4	5	5	6
1	2	5	5	3	4	3	2	6	5
1	1	1	1	1	1	1	1	1	1

format in $P_1 G$ (attribute edge-index)

• **Compressed Sparse Row - CSR**
3 indexed structures

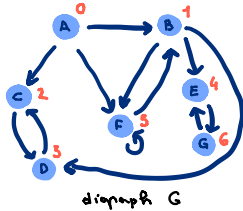
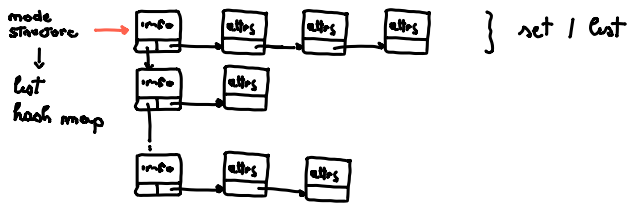
row $\in \mathbb{R}^m$ $[i_1, \dots, i_m]$
index for mode at j indicating where the set of modes incident to j starts
col_ind $\in \mathbb{R}^e$ indexed set of links $[v_1, \dots, v_K, \dots]$
 $(u, v_1), \dots, (u, v_K)$
val $\in \mathbb{R}^e$ weight or attribute functions for each link



col_ind = $[1, 2, 5, 3, 4, 5, 3, 2, 6, 1, 5, 4]$
row = $[0, 3, 6, 7, 8, 9, 11]$
mode = $[A, B, C, D, E, F, G] \rightsquigarrow \begin{matrix} A \rightarrow 0 & C \rightarrow 2 & E \rightarrow 4 \\ B \rightarrow 1 & D \rightarrow 3 & F \rightarrow 5 \end{matrix}$

Δ **ADJACENCY LIST** : more dynamic data structure w.r.t. CSR but same idea = store adjacent modes of u

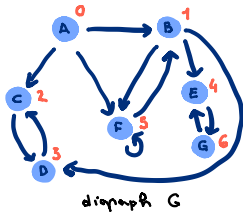
pointers instead of integer index



$A \rightarrow \{B, C, F\}$
 $B \rightarrow \{D, E, F\}$
 $C \rightarrow \{D\}$
 $D \rightarrow \{C\}$
 $E \rightarrow \{G\}$
 $F \rightarrow \{B, E\}$
 $G \rightarrow \{E\}$

Δ **EDGE LIST** : based on idea graph = set of links

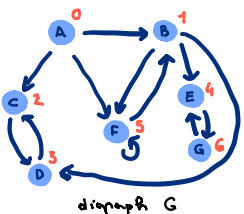
list of tuples $(u, v, \text{weight?})$



$[(A, B), (A, C), (A, F), (B, D), (B, E), (B, F), (C, D), (D, C), (E, G), (F, F), (F, B), (G, E)]$

Δ **INCIDENT MATRIX** : $A_I \in \mathbb{R}^{m \times e}$
rectangular matrix

$(u, v) \in E \Leftrightarrow A_I(u, (u, v)) = 1 \wedge A_I(v, (u, v)) = -1$ else 0
 $\{u, v\} \in E \Leftrightarrow A_I(u, (u, v)) = 1 \wedge A_I(v, (u, v)) = 1$ else 0
→ solve with self-loop



e_1 e_2 e_3 e_4 e_5 e_6 e_7 e_8 e_9 e_{10} e_{11} e_{12}
 $[(A, B), (A, C), (A, F), (B, D), (B, E), (B, F), (C, D), (D, C), (E, G), (F, F), (F, B), (G, E)]$

	e_1	e_2	e_3	e_4	e_5	e_6	e_7	e_8	e_9	e_{10}	e_{11}	e_{12}
A	1	1	1	0	0	0	0	0	0	0	0	0
B	-1	0	0	1	1	1	0	0	0	0	-1	0
C	0	-1	0	0	0	0	1	-1	0	0	0	0
D	0	0	0	-1	0	0	-1	1	0	0	0	0
E	0	0	0	0	-1	0	0	0	1	0	0	-1
F	0	0	-1	0	0	-1	0	0	0	1	1	0
G	0	0	0	0	0	0	0	0	-1	0	0	1